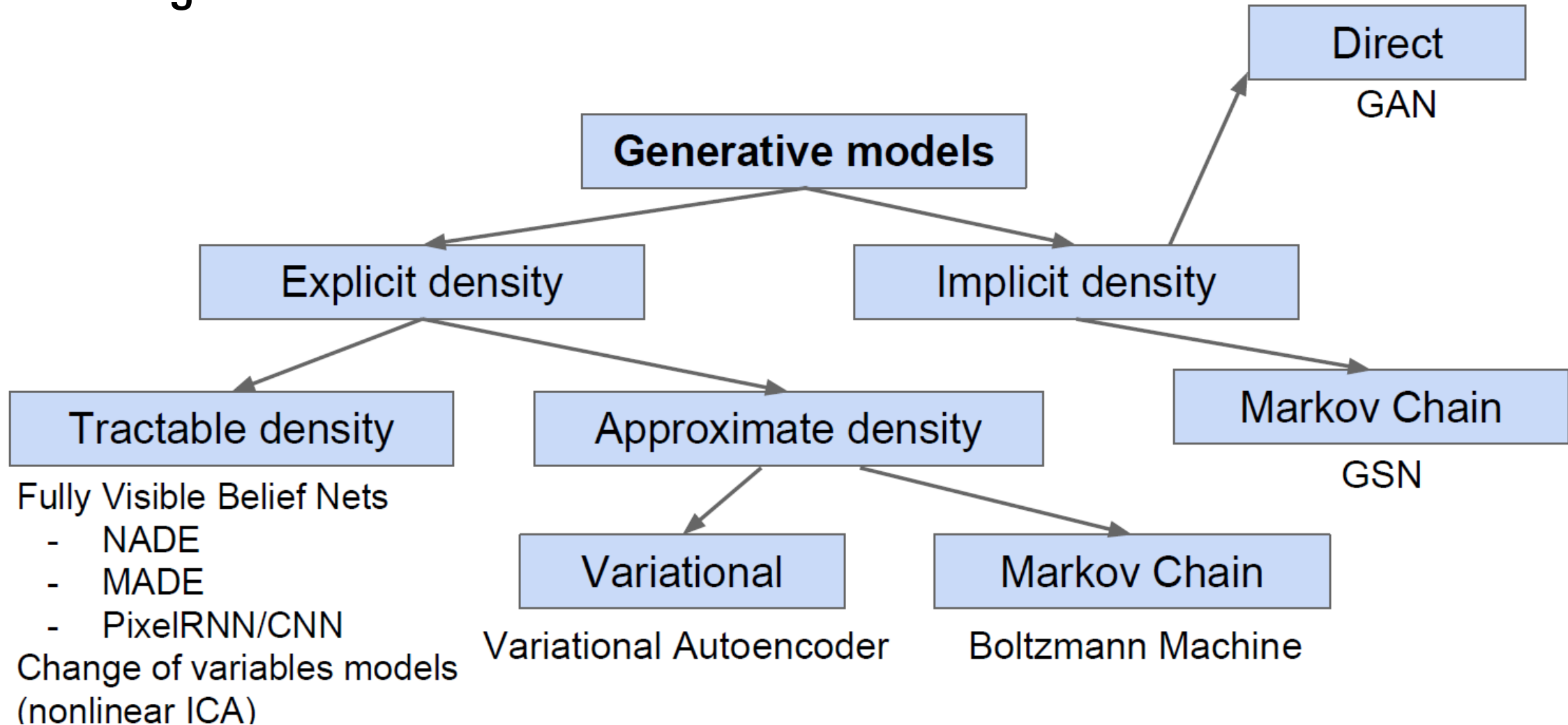


normalizing flows

stats403_deep_learning
spring_2025
lecture_9

9.1 bridge

a LOT of generative models



normalizing flow

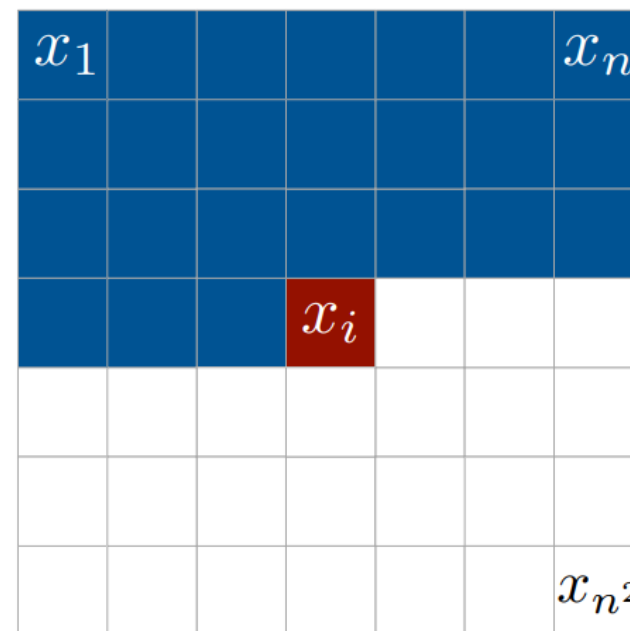
image credit: Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017,
available at <https://arxiv.org/pdf/1701.00160.pdf>
also see http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture13.pdf

pixelRNN

- to generate a pixel, we condition on all the previously generated

$$p(\mathbf{x}) = \prod_{i=1}^{n^2} p(x_i | x_1, \dots, x_{i-1})$$

- modeled by an RNN
- trained according to maximum likelihood



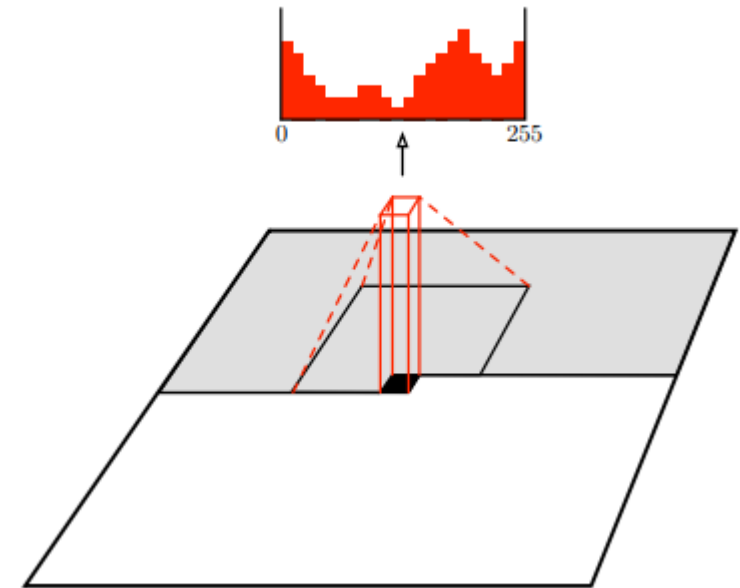
pixelCNN

- one can alternatively model dependency on previous pixels using a CNN over context region.

1	1	1	1	1
1	1	1	1	1
1	1	0	0	0
0	0	0	0	0
0	0	0	0	0

mask

```
def forward(self, x):  
    self.conv.weight.data *= self.mask  
    return self.conv(x)
```



pixelRNN / pixelCNN

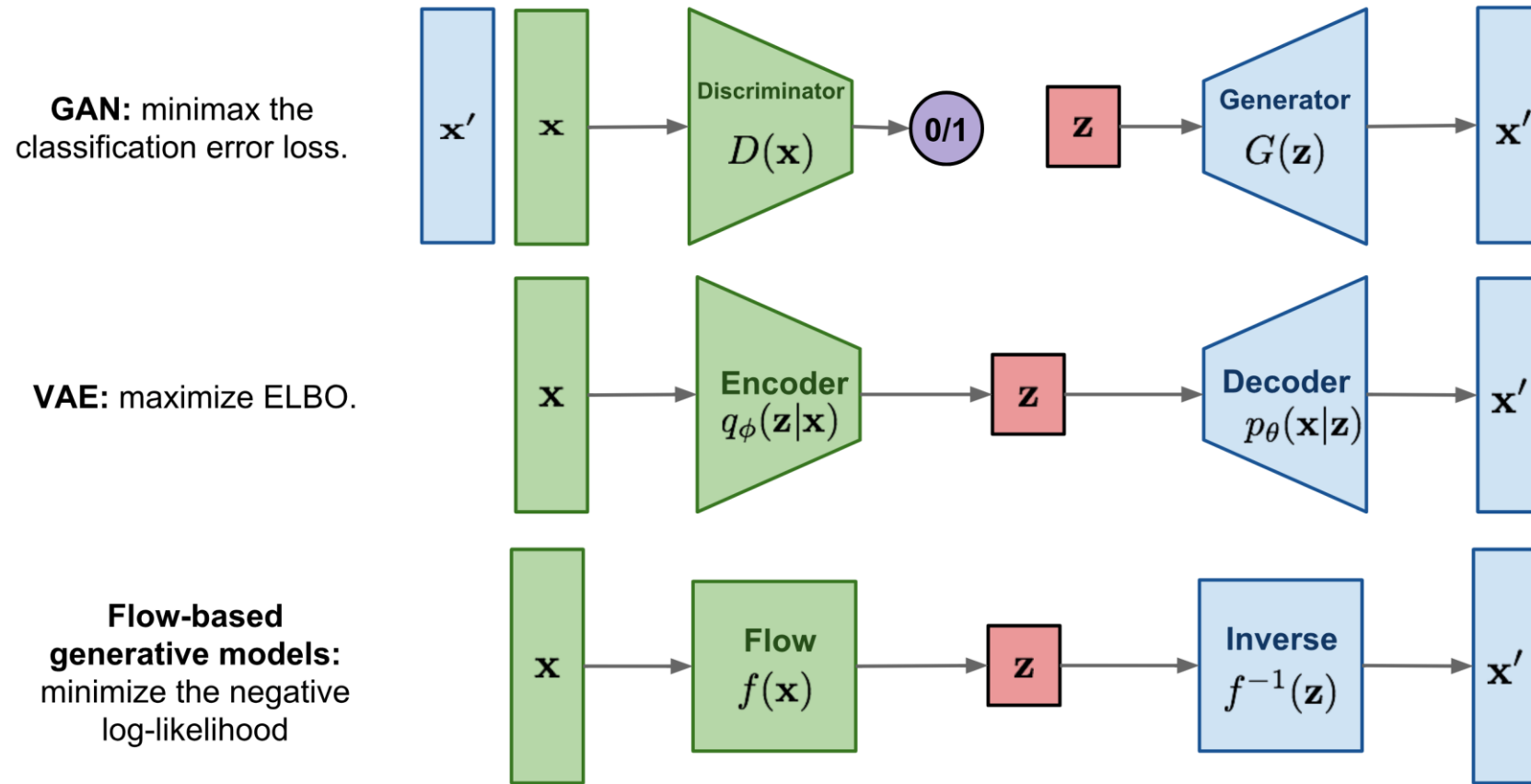
- can be used as the decoder in (V)AE
- produces good examples
- but is slow 🐱

9.2 normalizing flows

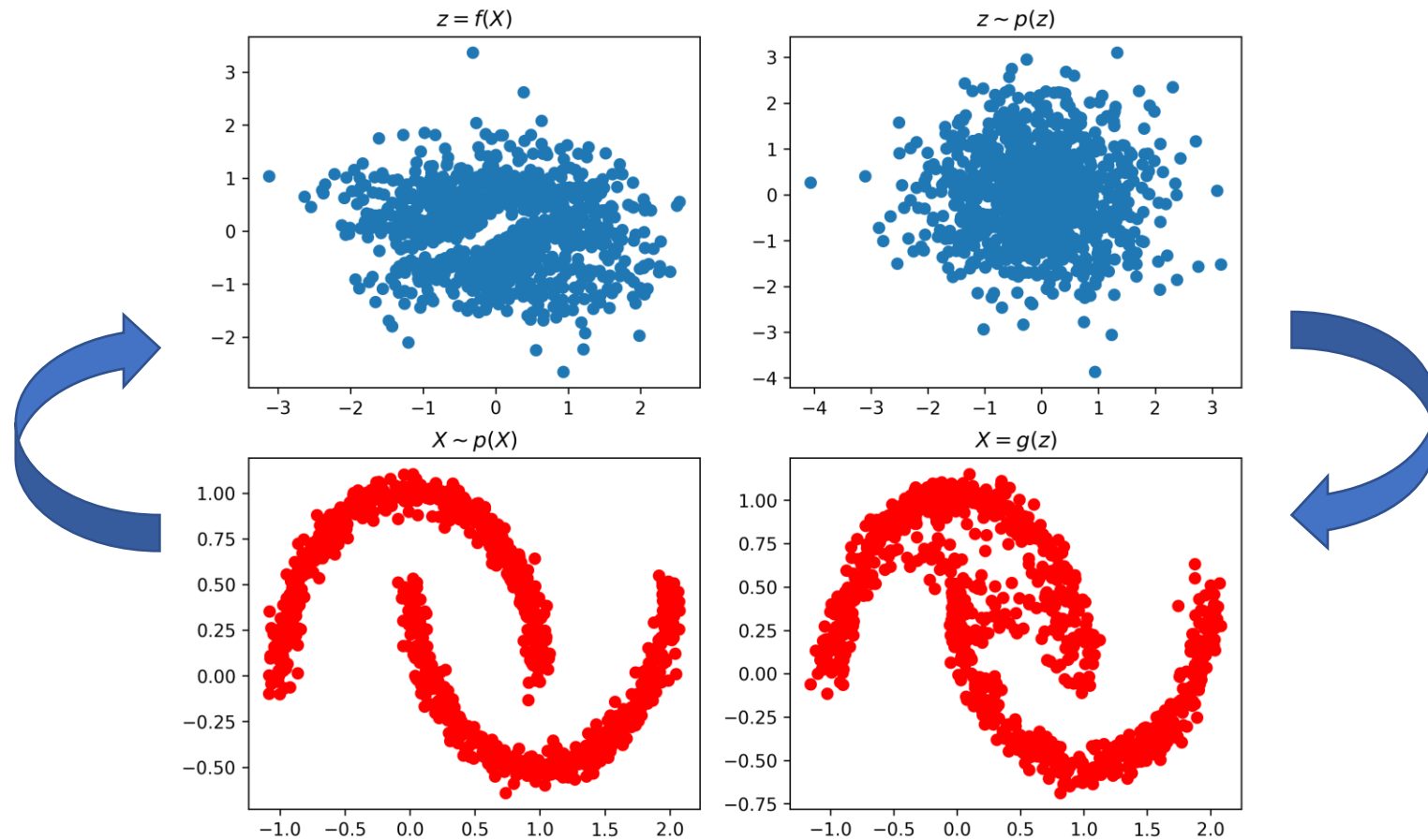
normalizing flow

- “a general mechanism for defining expressive probability distributions, only requiring the specification of a (usually simple) **base distribution** and a series of **bijective transformations**”
- **normalizing**: the change of variables gives a normalized density after applying invertible transformations
- **flow**: stack of simple transformations

comparison of generative models



normalizing flows



transformation

- consider

$$\mathbf{x} = T(\mathbf{u}) \quad \text{where} \quad \mathbf{u} \sim p_{\mathbf{u}}(\mathbf{u}).$$

The diagram illustrates the components of the transformation $\mathbf{x} = T(\mathbf{u})$. At the bottom, the set of parameters $\{\phi, \psi\}$ is labeled "parameters". Two blue arrows originate from this set: one points upwards to the transformation function T in the equation, and the other points upwards to the base distribution $p_{\mathbf{u}}(\mathbf{u})$ in the equation. The label "base distribution" is placed next to the arrow pointing to $p_{\mathbf{u}}(\mathbf{u})$.

T needs to be **diffomorphic**: T is invertible and both T and T^{-1} are differentiable.

transformation

$$\mathbf{x} = T(\mathbf{u}) \quad \text{where} \quad \mathbf{u} \sim p_{\mathbf{u}}(\mathbf{u}).$$

$$\left\{ \begin{array}{l} p_{\mathbf{x}}(\mathbf{x}) = p_{\mathbf{u}}(\mathbf{u}) |\det J_T(\mathbf{u})|^{-1} \\ p_{\mathbf{x}}(\mathbf{x}) = p_{\mathbf{u}}(T^{-1}(\mathbf{x})) |\det J_{T^{-1}}(\mathbf{x})| \end{array} \right.$$

composition

- consider $T_2 \circ T_1$

- its inverse is

$$(T_2 \circ T_1)^{-1} = T_1^{-1} \circ T_2^{-1}$$

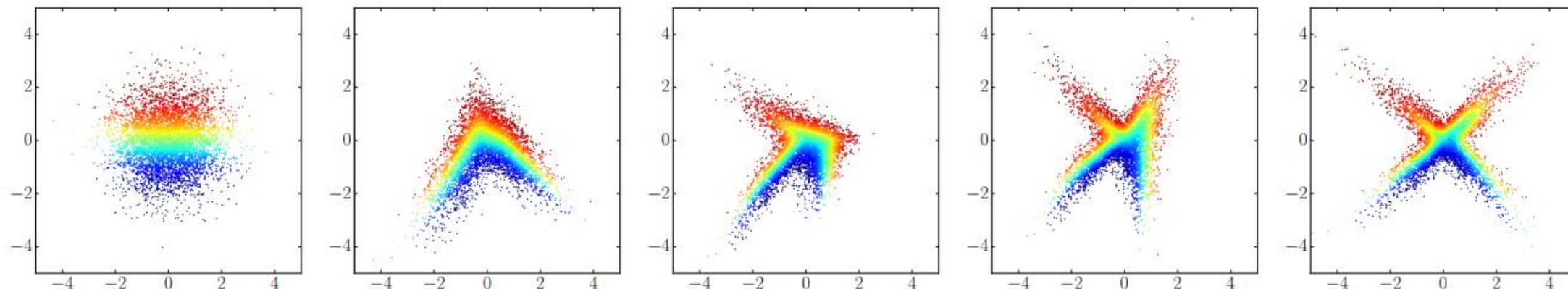
- its Jacobian is

$$\det J_{T_2 \circ T_1}(\mathbf{u}) = \det J_{T_2}(T_1(\mathbf{u})) \cdot \det J_{T_1}(\mathbf{u})$$

normalizing flow

- we can chain together multiple transformations $T_1, \dots, T_K$ to get

$$T = T_K \circ \dots \circ T_1$$



normalizing flows

- **sampling** from the model requires the ability to sample from the **base distribution** and to compute the **forward transformation**
- **evaluating** the model's density requires computing the **inverse transformation** and its **Jacobian determinant** and evaluating the **base density**

expressivity

- fact: for any pair of well-behaved distributions, **there exists** a diffeomorphism that can turn the base distribution into the target distribution
- why?

expressivity

- why? $p_{\mathbf{x}}(\mathbf{x}) = \prod_{i=1}^D p_{\mathbf{x}}(x_i \mid \mathbf{x}_{<i})$

define $F : \mathbf{x} \mapsto \mathbf{z} \in (0, 1)^D$

$$z_i = F_i(x_i, \mathbf{x}_{<i}) = \int_{-\infty}^{x_i} p_{\mathbf{x}}(x'_i \mid \mathbf{x}_{<i}) dx'_i = \Pr(x'_i \leq x_i \mid \mathbf{x}_{<i})$$

$$x_i = (F_i(\cdot, \mathbf{x}_{<i}))^{-1}(z_i)$$

Jacobian of F is lower-triangular: $\det J_F(\mathbf{x}) = \prod_{i=1}^D \frac{\partial F_i}{\partial x_i} = \prod_{i=1}^D p_{\mathbf{x}}(x_i \mid \mathbf{x}_{<i}) = p_{\mathbf{x}}(\mathbf{x})$

$$p_{\mathbf{z}}(\mathbf{z}) = p_{\mathbf{x}}(\mathbf{x}) |\det J_F(\mathbf{x})|^{-1} = p_{\mathbf{x}}(\mathbf{x}) |p_{\mathbf{x}}(\mathbf{x})|^{-1} = 1$$

fitting flows

- in order to fit a parametric flow model to data, we want to minimize the discrepancy between the flow model (which expresses a distribution) and the target distribution
- for instance, we can choose to minimize the KL divergence between the flow model and the target distribution

forward KL divergence

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= D_{\text{KL}} \left[\overset{\text{target}}{p_{\mathbf{x}}^*(\mathbf{x})} \parallel \overset{\text{flow-based model}}{p_{\mathbf{x}}(\mathbf{x}; \boldsymbol{\theta})} \right] \\ &= -\mathbb{E}_{p_{\mathbf{x}}^*(\mathbf{x})} [\log p_{\mathbf{x}}(\mathbf{x}; \boldsymbol{\theta})] + \text{const.} \\ &= -\mathbb{E}_{p_{\mathbf{x}}^*(\mathbf{x})} [\log p_{\mathbf{u}}(T^{-1}(\mathbf{x}; \boldsymbol{\phi}); \boldsymbol{\psi}) + \log |\det J_{T^{-1}}(\mathbf{x}; \boldsymbol{\phi})|] + \text{const.}\end{aligned}$$

$\boldsymbol{\phi}$: parameters of T
 $\boldsymbol{\psi}$: parameters of $p_{\mathbf{u}}(\mathbf{u})$

- if we have samples,

$$\mathcal{L}(\boldsymbol{\theta}) \approx -\frac{1}{N} \sum_{n=1}^N \log p_{\mathbf{u}}(T^{-1}(\mathbf{x}_n; \boldsymbol{\phi}); \boldsymbol{\psi}) + \log |\det J_{T^{-1}}(\mathbf{x}_n; \boldsymbol{\phi})| + \text{const.}$$

forward KL divergence



$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= D_{\text{KL}} \left[\overset{\text{target}}{p_{\mathbf{x}}^*(\mathbf{x})} \parallel \overset{\text{flow-based model}}{p_{\mathbf{x}}(\mathbf{x}; \boldsymbol{\theta})} \right] \\ &= -\mathbb{E}_{p_{\mathbf{x}}^*(\mathbf{x})} [\log p_{\mathbf{x}}(\mathbf{x}; \boldsymbol{\theta})] + \text{const.} \\ &= -\mathbb{E}_{p_{\mathbf{x}}^*(\mathbf{x})} [\log p_{\mathbf{u}}(T^{-1}(\mathbf{x}; \boldsymbol{\phi}); \boldsymbol{\psi}) + \log |\det J_{T^{-1}}(\mathbf{x}; \boldsymbol{\phi})|] + \text{const.}\end{aligned}$$

- if we have samples (**from where?**),

$$\mathcal{L}(\boldsymbol{\theta}) \approx -\frac{1}{N} \sum_{n=1}^N \log p_{\mathbf{u}}(T^{-1}(\mathbf{x}_n; \boldsymbol{\phi}); \boldsymbol{\psi}) + \log |\det J_{T^{-1}}(\mathbf{x}_n; \boldsymbol{\phi})| + \text{const.}$$

forward KL divergence and ML

- we can get the gradients

$$\nabla_{\phi} \mathcal{L}(\theta) \approx -\frac{1}{N} \sum_{n=1}^N \nabla_{\phi} \log p_{\mathbf{u}}(T^{-1}(\mathbf{x}_n; \phi); \psi) + \nabla_{\phi} \log |\det J_{T^{-1}}(\mathbf{x}_n; \phi)|$$

$$\nabla_{\psi} \mathcal{L}(\theta) \approx -\frac{1}{N} \sum_{n=1}^N \nabla_{\psi} \log p_{\mathbf{u}}(T^{-1}(\mathbf{x}_n; \phi); \psi).$$

reverse KL divergence

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= \overset{\text{flow-based model}}{D_{\text{KL}}} [p_{\mathbf{x}}(\mathbf{x}; \boldsymbol{\theta}) \parallel \overset{\text{target}}{p_{\mathbf{x}}^*(\mathbf{x})}] \\ &= \mathbb{E}_{p_{\mathbf{x}}(\mathbf{x}; \boldsymbol{\theta})} [\log p_{\mathbf{x}}(\mathbf{x}; \boldsymbol{\theta}) - \log p_{\mathbf{x}}^*(\mathbf{x})] \\ &= \mathbb{E}_{p_{\mathbf{u}}(\mathbf{u}; \boldsymbol{\psi})} [\log p_{\mathbf{u}}(\mathbf{u}; \boldsymbol{\psi}) - \log |\det J_T(\mathbf{u}; \boldsymbol{\phi})| - \log p_{\mathbf{x}}^*(T(\mathbf{u}; \boldsymbol{\phi}))].\end{aligned}$$

- this is useful when we cannot sample from the target distribution but can evaluate it (up to a multiplicative constant)

reverse KL divergence

- assume that we can evaluate $p_x^*(x)$ up to a constant, i.e.,
$$p_x^*(\mathbf{x}) = \tilde{p}_x(\mathbf{x})/C$$

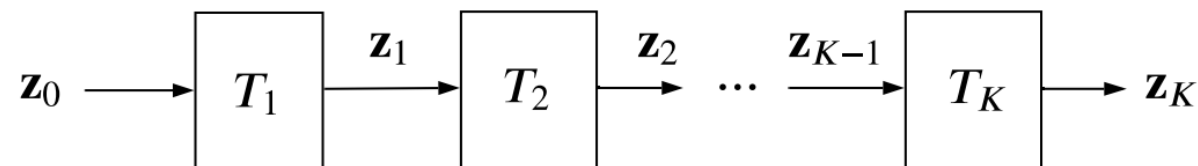
$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}_{p_{\mathbf{u}}(\mathbf{u}; \boldsymbol{\psi})} [\log p_{\mathbf{u}}(\mathbf{u}; \boldsymbol{\psi}) - \log |\det J_T(\mathbf{u}; \boldsymbol{\phi})| - \log \tilde{p}_{\mathbf{x}}(T(\mathbf{u}; \boldsymbol{\phi}))] + \text{const.}$$

$$\nabla_{\boldsymbol{\phi}} \mathcal{L}(\boldsymbol{\theta}) \approx -\frac{1}{N} \sum_{n=1}^N \nabla_{\boldsymbol{\phi}} \log |\det J_T(\mathbf{u}_n; \boldsymbol{\phi})| + \nabla_{\boldsymbol{\phi}} \log \tilde{p}_{\mathbf{x}}(T(\mathbf{u}_n; \boldsymbol{\phi})).$$

alternative divergences

- discussion: what other measures of discrepancy can be used for fitting the flow model?

constructing flows



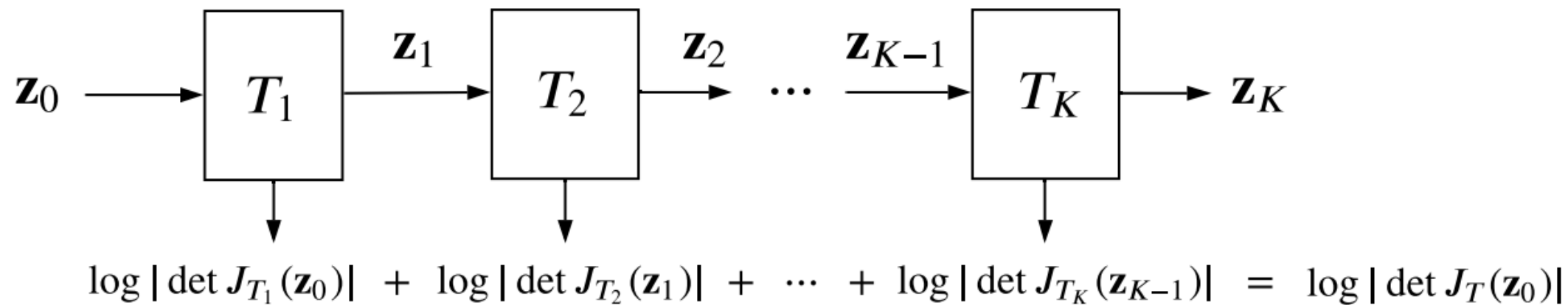
$$T = T_K \circ \cdots \circ T_1.$$

$$\mathbf{z}_k = T_k(\mathbf{z}_{k-1}) \quad \text{for } k = 1, \dots, K,$$

$$\mathbf{z}_{k-1} = T_k^{-1}(\mathbf{z}_k) \quad \text{for } k = K, \dots, 1,$$

$$\log |\det J_T(\mathbf{z}_0)| = \log \left| \prod_{k=1}^K \det J_{T_k}(\mathbf{z}_{k-1}) \right| = \sum_{k=1}^K \log |\det J_{T_k}(\mathbf{z}_{k-1})|.$$

constructing flows



constructing flows

- in practice, either T_k or T_k^{-1} is implemented using a model (usually a neural network)
 - from now on, we use f_ϕ to denote such neural network where ϕ denotes the set of parameters (we omit the dependence on k since we consider a generic block in the chain)
- we need to make sure that the model is invertible and has a tractable Jacobian determinant
 - in applications, it may be important to make the inverse easily implementable
 - it is also important to have efficient approach to calculation of the Jacobian

taxonomy

Autoregressive flows

Transformer type:

- Affine
- Combination-based
- Integration-based
- Spline-based

Conditioner type:

- Recurrent
 - Masked
 - Coupling layer
-

Linear flows

Permutations

Decomposition-based:

- PLU
- QR

Orthogonal:

- Exponential map
 - Cayley map
 - Householder
-

Residual flows

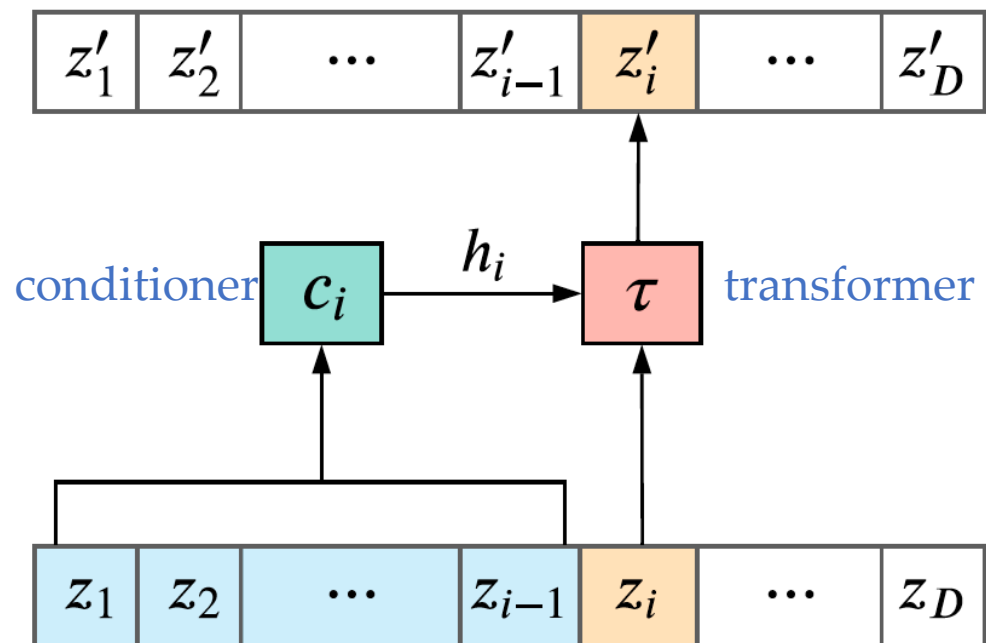
Contractive residual

Based on matrix determinant lemma:

- Planar
 - Sylvester
 - Radial
-

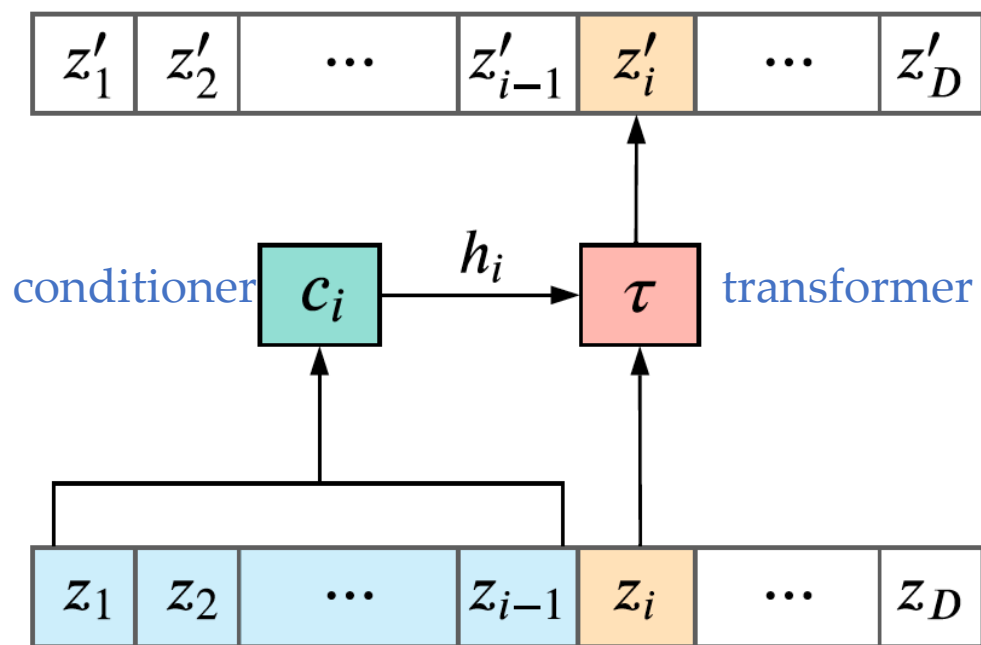
case study 1: autoregressive flows

$$z'_i = \tau(z_i; \mathbf{h}_i) \quad \text{where} \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i}),$$

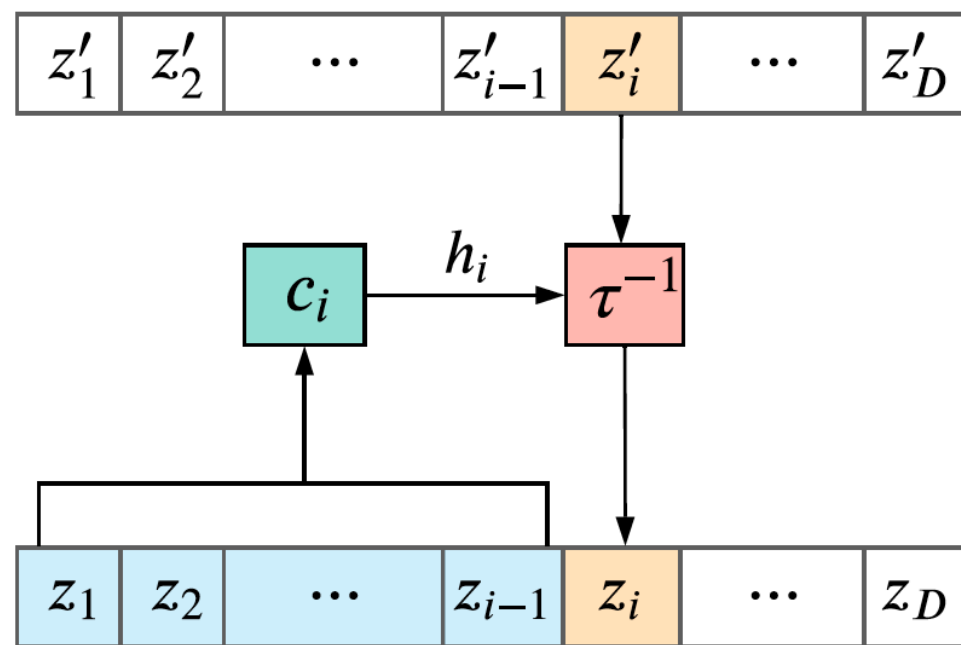


case study 1: autoregressive flows

$$z'_i = \tau(z_i; \mathbf{h}_i) \quad \text{where} \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i}), \quad \longleftrightarrow \quad z_i = \tau^{-1}(z'_i; \mathbf{h}_i) \quad \text{where} \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i}).$$



(a) Forward



(b) Inverse

case study 1: autoregressive flows

$$z'_i = \tau(z_i; \mathbf{h}_i) \quad \text{where} \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i}), \quad \longleftrightarrow \quad z_i = \tau^{-1}(z'_i; \mathbf{h}_i) \quad \text{where} \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i}).$$

$$J_{f_\phi}(\mathbf{z}) = \begin{bmatrix} \frac{\partial \tau}{\partial z_1}(z_1; \mathbf{h}_1) & & \mathbf{0} \\ & \ddots & \\ \mathbf{L}(\mathbf{z}) & & \frac{\partial \tau}{\partial z_D}(z_D; \mathbf{h}_D) \end{bmatrix}.$$

case study 1: autoregressive flows

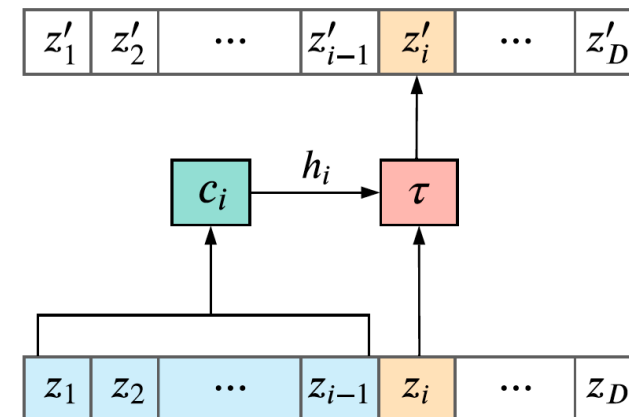
$$z'_i = \tau(z_i; \mathbf{h}_i) \quad \text{where} \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i}), \quad \longleftrightarrow \quad z_i = \tau^{-1}(z'_i; \mathbf{h}_i) \quad \text{where} \quad \mathbf{h}_i = c_i(\mathbf{z}_{<i}).$$

$$J_{f_\phi}(\mathbf{z}) = \begin{bmatrix} \frac{\partial \tau}{\partial z_1}(z_1; \mathbf{h}_1) & & \mathbf{0} \\ & \ddots & \\ \mathbf{L}(\mathbf{z}) & & \frac{\partial \tau}{\partial z_D}(z_D; \mathbf{h}_D) \end{bmatrix}.$$

$$\log |\det J_{f_\phi}(\mathbf{z})| = \log \left| \prod_{i=1}^D \frac{\partial \tau}{\partial z_i}(z_i; \mathbf{h}_i) \right| = \sum_{i=1}^D \log \left| \frac{\partial \tau}{\partial z_i}(z_i; \mathbf{h}_i) \right|$$

case study 1: autoregressive flows

- simplest transformer: **affine transformers**



$$\tau(z_i; \mathbf{h}_i) = \alpha_i z_i + \beta_i \quad \text{where} \quad \mathbf{h}_i = \{\alpha_i, \beta_i\}.$$

- invertible as long as $\alpha_i \neq 0$, say $\alpha_i = \exp(\tilde{\alpha}_i)$

$$\log |\det J_{f_\phi}(\mathbf{z})| = \sum_{i=1}^D \log |\alpha_i| = \sum_{i=1}^D \tilde{\alpha}_i.$$

case study 1: autoregressive flows

other examples of transformers:

- combination-based transformers

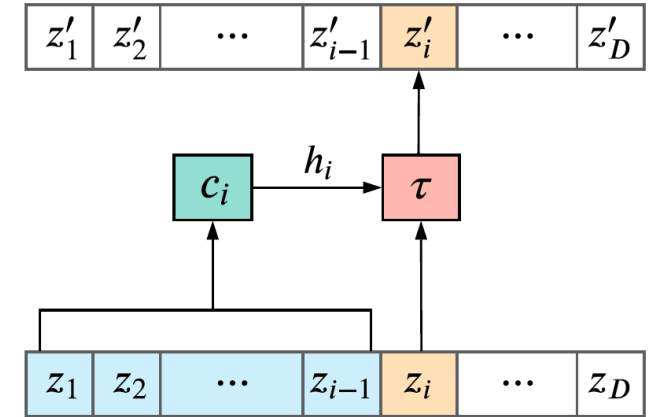
$$\tau(z_i; \mathbf{h}_i) = w_{i0} + \sum_{k=1}^K w_{ik} \sigma(\alpha_{ik} z_i + \beta_{ik}) \quad \text{where} \quad \mathbf{h}_i = \{w_{i0}, \dots, w_{iK}, \alpha_{ik}, \beta_{ik}\},$$

with positive coefficients and monotonically increasing nonlinearity

- integration-based transformers

$$\tau(z_i; \mathbf{h}_i) = \int_0^{z_i} g(z; \boldsymbol{\alpha}_i) dz + \beta_i \quad \text{where} \quad \mathbf{h}_i = \{\boldsymbol{\alpha}_i, \beta_i\},$$

where g is positive valued



case study 1: autoregressive flows

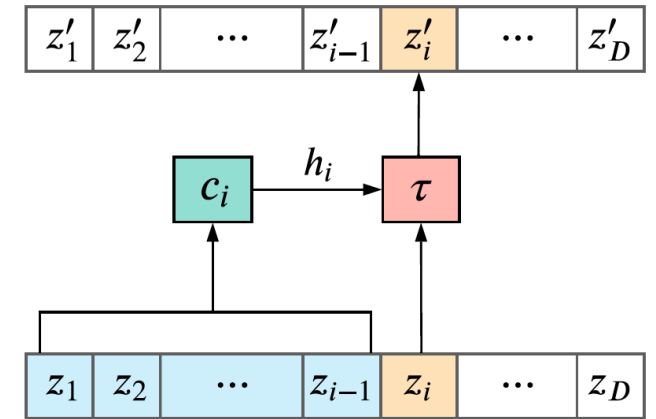
- conditioner

- recurrent conditioner

$$\mathbf{h}_i = c(\mathbf{s}_i) \quad \text{where} \quad \begin{aligned} \mathbf{s}_1 &= \text{initial state} \\ \mathbf{s}_i &= \text{RNN}(z_{i-1}, \mathbf{s}_{i-1}) \text{ for } i > 1 \end{aligned}$$

- masked conditioner

□ take an input $\mathbf{z}$ and output the entire sequence $\mathbf{h}_1, \dots, \mathbf{h}_D$ in one pass, e.g., take an arbitrary neural network and remove connections until there is no path from input z_i to outputs $\mathbf{h}_1, \dots, \mathbf{h}_i$

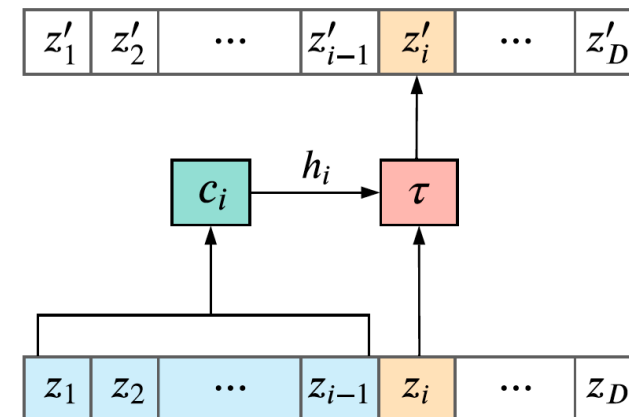


case study 1: autoregressive flows

- conditioner

- coupling layers: choose an index d (e.g., $d = D/2$)
 - Parameters $(\mathbf{h}_1, \dots, \mathbf{h}_d)$ are constants, i.e. not a function of $\mathbf{z}$.
 - Parameters $(\mathbf{h}_{d+1}, \dots, \mathbf{h}_D)$ are functions of $\mathbf{z}_{\leq d}$ only, i.e. they don't depend on $\mathbf{z}_{>d}$.
- for instance, we can fix $\tau(\cdot; \mathbf{h}_1), \dots, \tau(\cdot; \mathbf{h}_d)$ to the identity function

$$\begin{array}{ccc}
 \mathbf{z}'_{\leq d} = \mathbf{z}_{\leq d} & & \mathbf{z}_{\leq d} = \mathbf{z}'_{\leq d} \\
 (\mathbf{h}_{d+1}, \dots, \mathbf{h}_D) = F(\mathbf{z}_{\leq d}) & \longleftrightarrow & (\mathbf{h}_{d+1}, \dots, \mathbf{h}_D) = F(\mathbf{z}_{\leq d}) \\
 \mathbf{z}'_i = \tau(\mathbf{z}_i; \mathbf{h}_i) \text{ for } i > d. & & \mathbf{z}_i = \tau^{-1}(\mathbf{z}'_i; \mathbf{h}_i) \text{ for } i > d.
 \end{array}$$



case study 1: autoregressive flows

- conditioner

- coupling layers: choose an index d (e.g., $d = D/2$)
 - Parameters $(\mathbf{h}_1, \dots, \mathbf{h}_d)$ are constants, i.e. not a function of $\mathbf{z}$.
 - Parameters $(\mathbf{h}_{d+1}, \dots, \mathbf{h}_D)$ are functions of $\mathbf{z}_{\leq d}$ only, i.e. they don't depend on $\mathbf{z}_{>d}$.
- for instance, we can fix $\tau(\cdot; \mathbf{h}_1), \dots, \tau(\cdot; \mathbf{h}_d)$ to the identity function

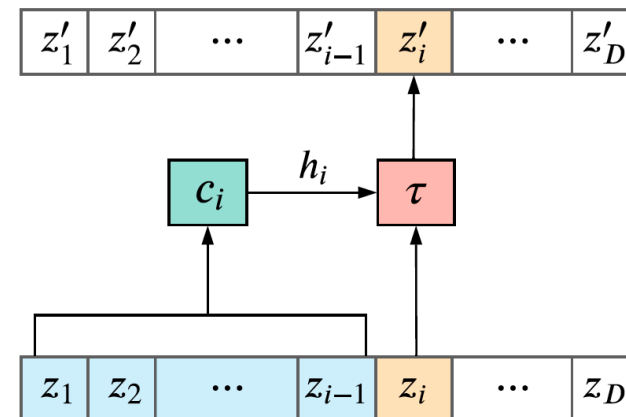
$$\mathbf{z}'_{\leq d} = \mathbf{z}_{\leq d}$$

$$(\mathbf{h}_{d+1}, \dots, \mathbf{h}_D) = F(\mathbf{z}_{\leq d})$$

$$\mathbf{z}'_i = \tau(\mathbf{z}_i; \mathbf{h}_i) \text{ for } i > d.$$

$$J_{f_\phi} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{A} & \mathbf{D} \end{bmatrix}$$

the diagonal elements of $\mathbf{D}$ are equal to the derivatives of the transformers $\tau(\cdot; \mathbf{h}_{d+1}), \dots, \tau(\cdot; \mathbf{h}_D)$

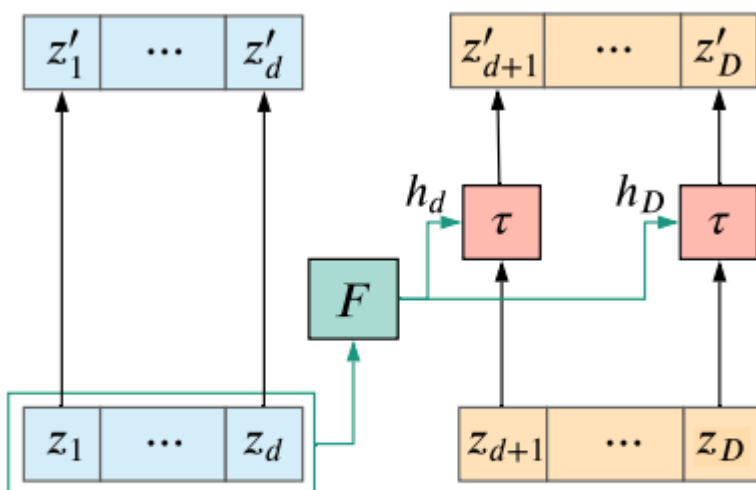
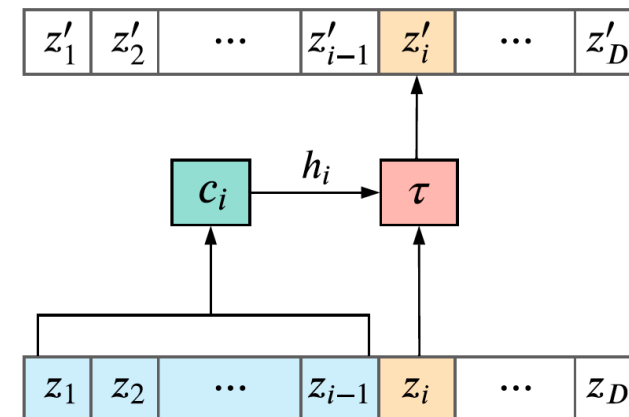


case study 1: autoregressive flows

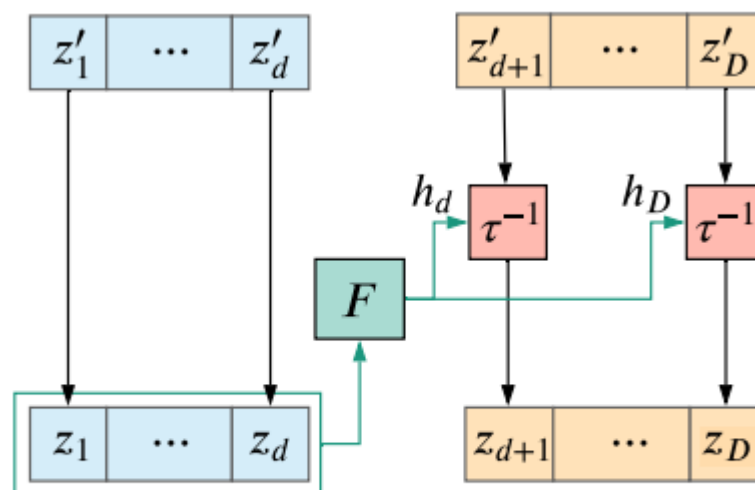
- conditioner

- coupling layers: choose an index d (e.g., $d = D/2$)

- Parameters $(h_1, \dots, h_d)$ are constants, i.e. not a function of $\mathbf{z}$.
- Parameters $(h_{d+1}, \dots, h_D)$ are functions of $\mathbf{z}_{\leq d}$ only, i.e. they don't depend on $\mathbf{z}_{>d}$.



(a) Forward



(b) Inverse

example: real-valued non-volume-preserving (real NVP) flows

- affine transformer + coupling conditioner

$$\begin{aligned} y_{1:d} &= x_{1:d} \\ y_{d+1:D} &= x_{d+1:D} \odot \exp(s(x_{1:d})) + t(x_{1:d}) \end{aligned} \Leftrightarrow \begin{cases} x_{1:d} &= y_{1:d} \\ x_{d+1:D} &= (y_{d+1:D} - t(y_{1:d})) \odot \exp(-s(y_{1:d})) \end{cases}$$

NNs

$$\frac{\partial y}{\partial x^T} = \begin{bmatrix} \mathbb{I}_d & 0 \\ \frac{\partial y_{d+1:D}}{\partial x_{1:d}^T} & \text{diag}(\exp[s(x_{1:d})]) \end{bmatrix}$$

- <https://colab.research.google.com/drive/1-ajio9u7xc2Mh1bGyn9id6JDpWYifZEI?usp=sharing>

(optional) case study 2: linear flows

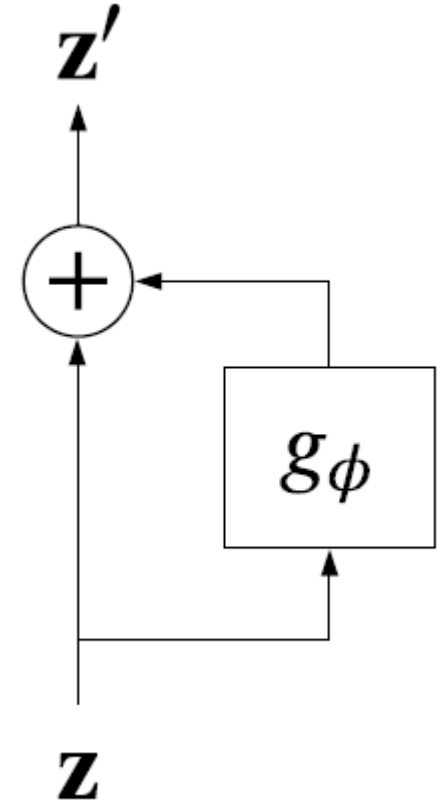
$$\mathbf{z}' = \mathbf{W}\mathbf{z}$$

- better expressivity
- permutation as a special case
 - The order of the input will determine the set of distributions the model can represent. If we can permute the input, then we can change the order.
- beyond permutation: parametrize the matrix $\mathbf{W}$

(optional) case study 3: residual flows

$$\mathbf{z}' = \mathbf{z} + g_{\phi}(\mathbf{z})$$

- similar to ResNet
- if g_{ϕ} is contractive, then the transformation is invertible



(optional) case study 3: residual flows

- in general, it is difficult to calculate Jacobian for residual flows
- the matrix determinant lemma

$$\det(\mathbf{A} + \mathbf{V}\mathbf{W}^\top) = \det(\mathbf{I} + \mathbf{W}^\top \mathbf{A}^{-1} \mathbf{V}) \det \mathbf{A}$$

planar flow (single hidden unit)
hidden $\mathbf{z}' = \mathbf{z} + \mathbf{v}\sigma(\mathbf{w}^\top \mathbf{z} + b)$

$$J_{f_\phi}(\mathbf{z}) = \mathbf{I} + \sigma'(\mathbf{w}^\top \mathbf{z} + b) \mathbf{v}\mathbf{w}^\top$$

$$\det J_{f_\phi}(\mathbf{z}) = 1 + \sigma'(\mathbf{w}^\top \mathbf{z} + b) \mathbf{w}^\top \mathbf{v}$$

Sylvester flow (multiple
 $\mathbf{z}' = \mathbf{z} + \mathbf{V}\sigma(\mathbf{W}^\top \mathbf{z} + \mathbf{b})$

$$J_{f_\phi}(\mathbf{z}) = \mathbf{I} + \mathbf{V}\mathbf{S}(\mathbf{z})\mathbf{W}^\top$$

$$\det J_{f_\phi}(\mathbf{z}) = \det(\mathbf{I} + \mathbf{S}(\mathbf{z})\mathbf{W}^\top \mathbf{V})$$

BN between transformations (e.g., $T_k \circ \text{BN} \circ T_{k-1}$)

$$\text{BN}(\mathbf{z}) = \boldsymbol{\alpha} \odot \frac{\mathbf{z} - \hat{\boldsymbol{\mu}}}{\sqrt{\hat{\boldsymbol{\sigma}}^2 + \boldsymbol{\epsilon}}} + \boldsymbol{\beta}, \quad \text{BN}^{-1}(\mathbf{z}') = \hat{\boldsymbol{\mu}} + \frac{\mathbf{z}' - \boldsymbol{\beta}}{\boldsymbol{\alpha}} \odot \sqrt{\hat{\boldsymbol{\sigma}}^2 + \boldsymbol{\epsilon}}$$

$$\det J_{\text{BN}}(\mathbf{z}) = \prod_{i=1}^D \frac{\alpha_i}{\sqrt{\hat{\sigma}_i^2 + \epsilon_i}}$$

Thank you!

Reference

- Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved training of Wasserstein GANs. *NIPS*.
- Papamakarios, G., Nalisnick, E., Rezende, D. J., Mohamed, S., & Lakshminarayanan, B. (2021). Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57), 1-64.
- <https://gebob19.github.io/normalizing-flows/>
- https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/

